

Анализ датасета Netflix 2025:User Behavior Dataset (210K+ Records)

Цель: Отработать навыки проектирования DWH на реальном датасете: спроектировать и исследовать реляционную структуру данных, автоматизировать ETL-процессы и реализовать комплекс бизнес-запросов.

Датасет: [Netflix 2025: User Behavior Dataset](#) — 210 000+ записей и 6 связанных таблиц.

Стек: · PostgreSQL · SQLAlchemy · DBeaver · Python · Jupyter Notebook · Pandas · Matplotlib · Seaborn

Структура анализа

1. Развёртывание DWH и изучение ER-схемы в DBeaver
2. Анализ географии притока аудитории
3. Месячная динамика активности пользователей (MAU)
4. Конверсия поиска по устройствам
5. Скользящее среднее 7 дней
6. ТОП-10 жанров
7. Когортный анализ удержания пользователей
8. Темп роста MoM
9. ТОП-3 фильма внутри каждого жанра
10. Финансовый профиль тарифов
11. Влияние качества трансляции на оценки
12. Анализ оттока пользователей

```
In [1]: import os
print(os.getcwd())
```

```
/Users/elizaveta/netflix_analytics/netflix_analytics
```

```
In [2]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sqlalchemy import create_engine

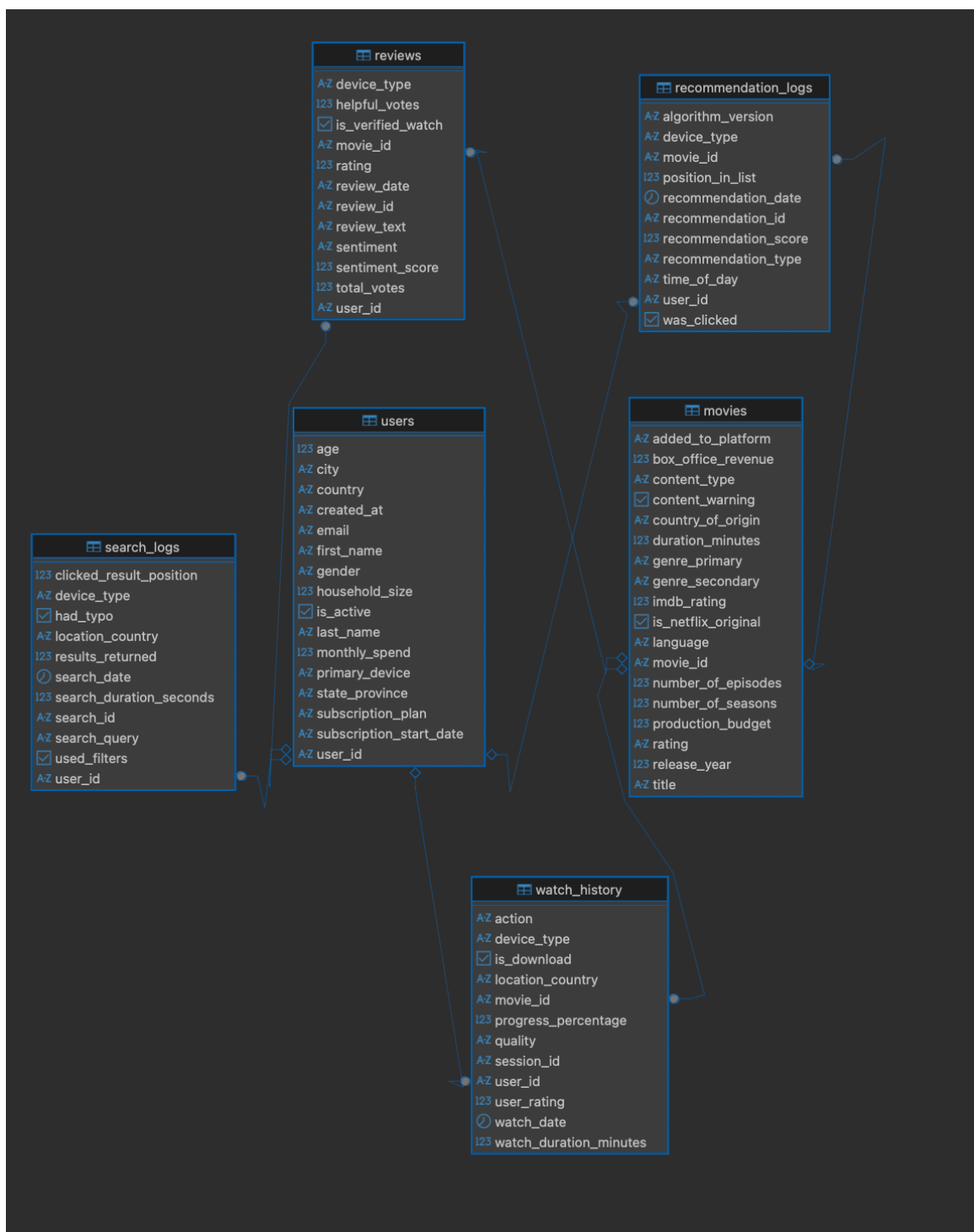
sns.set_theme(style="whitegrid")
engine = create_engine('postgresql://postgres:1234@localhost:5432/n')
tables = ["users", "movies", "watch_history", "reviews", "search_lo
dfs = {name: pd.read_csv(f"{name}.csv") for name in tables}
dfs["users"] = dfs["users"].drop_duplicates()
```

```
dfs["reviews"]["review_text"] = dfs["reviews"]["review_text"].fillna  
date_cols = {  
    "users": "subscription_start_date",  
    "watch_history": "watch_date",  
    "search_logs": "search_date",  
    "recommendation_logs": "recommendation_date"  
}  
for t_name, col in date_cols.items():  
    if col in dfs[t_name].columns:  
        dfs[t_name][col] = pd.to_datetime(dfs[t_name][col], errors=  
for name, df in dfs.items():  
    df.to_sql(name, engine, if_exists="replace", index=False)
```

1. Развёртывание DWH и изучение ER-схемы в DBeaver

Изучим, по каким первичным ключам соединены таблицы и какие столбцы входят в каждую из них.

```
In [3]: from IPython.display import Image, display  
display(Image(filename='ER таблица.png', width=800))
```



2. Анализ географии притока аудитории

Проанализируем географию пользователей, остортируем по убыванию пользователей и составим рейтинг по штатам/провинциям.

```
In [4]: geography = pd.read_sql("""
        SELECT
        country,
        state_province,
        COUNT(user_id) AS total_new_users
```

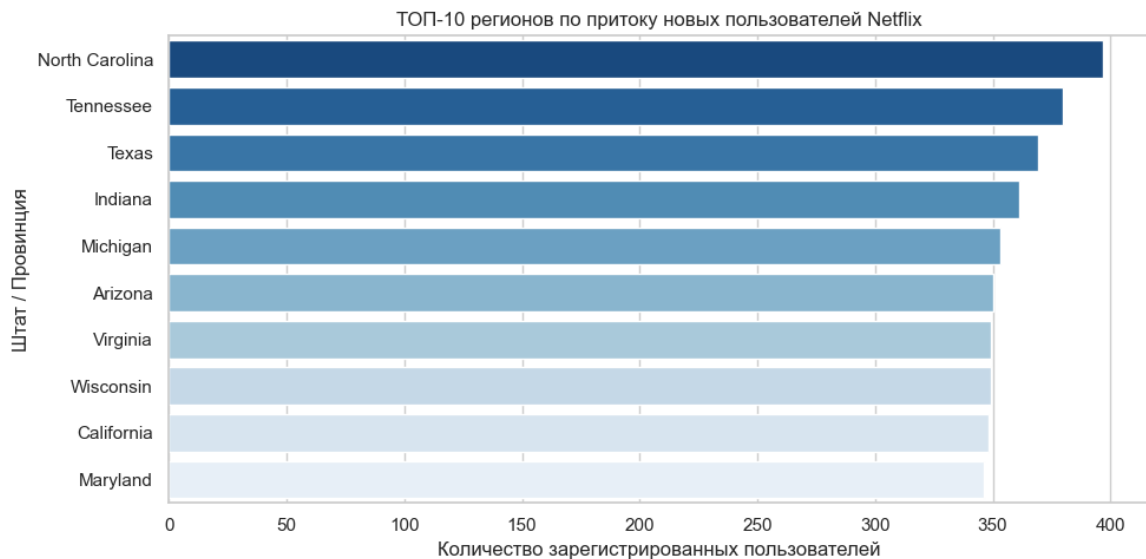
```
FROM users
GROUP BY country, state_province
ORDER BY total_new_users DESC;
""" , engine)
geography
```

Out [4]:

	country	state_province	total_new_users
0	USA	North Carolina	397
1	USA	Tennessee	380
2	USA	Texas	369
3	USA	Indiana	361
4	USA	Michigan	353
5	USA	Arizona	350
6	USA	Virginia	349
7	USA	Wisconsin	349
8	USA	California	348
9	USA	Maryland	346
10	USA	Illinois	343
11	USA	Georgia	342
12	USA	Pennsylvania	342
13	USA	Ohio	341
14	USA	New Jersey	341
15	USA	New York	340
16	USA	Florida	340
17	USA	Missouri	339
18	USA	Massachusetts	339
19	Canada	Nova Scotia	336
20	USA	Washington	324
21	Canada	Ontario	312
22	Canada	Quebec	310
23	Canada	Prince Edward Island	306
24	Canada	Alberta	300
25	Canada	British Columbia	298
26	Canada	Newfoundland and Labrador	298
27	Canada	Manitoba	289
28	Canada	New Brunswick	283
29	Canada	Saskatchewan	275

In [5]: `plt.figure(figsize=(10,5))`

```
sns.barplot(
    data=geography.head(10),
    x='total_new_users',
    y='state_province',
    palette='Blues_r',
    hue='state_province',
    legend=False
)
plt.title('ТОП-10 регионов по притоку новых пользователей Netflix')
plt.xlabel('Количество зарегистрированных пользователей')
plt.ylabel('Штат / Провинция')
plt.tight_layout()
plt.show()
```



Вывод: Новая аудитория живет в США (подавляющее большинство) и Канаде. Новые пользователи преимущественно живут в Северной Каролине, Теннесси и Техасе, а многомиллионный штат Калифорния занимает лишь 9 место. Значит мегаполисы достигли точки плато, следовательно основным притоком аудитории являются регионы. Рекомендация: перенаправить часть маркетингового бюджета в другие штаты, со средней плотностью населения.

3. Месячная динамика активности пользователей (MAU)

Найдем, в какой период падает активность просмотра контента

```
In [6]: mau = pd.read_sql("""
        SELECT COUNT(DISTINCT user_id) AS active_users,
               SUM(watch_duration_minutes) AS total_minutes,
               TO_CHAR(watch_date::TIMESTAMP, 'YYYY-MM') AS period
               ROUND((SUM(watch_duration_minutes)::NUMERIC/ COUNT(
        FROM watch_history
        GROUP BY period
```

```
ORDER BY period ASC;  
""", engine)  
mau
```

Out[6]:

	active_users	total_minutes	period	average_sum
0	3372	256900.9	2024-01	76.19
1	3384	242275.2	2024-02	71.59
2	3380	250797.5	2024-03	74.20
3	3387	244527.7	2024-04	72.20
4	3551	270953.5	2024-05	76.30
5	3445	261397.6	2024-06	75.88
6	3398	261720.3	2024-07	77.02
7	3551	269303.1	2024-08	75.84
8	3323	251892.0	2024-09	75.80
9	3434	255226.5	2024-10	74.32
10	3331	236574.4	2024-11	71.02
11	3539	268562.2	2024-12	75.89
12	3424	255829.7	2025-01	74.72
13	3118	223745.4	2025-02	71.76
14	3403	257051.5	2025-03	75.54
15	3373	251757.0	2025-04	74.64
16	3438	260526.5	2025-05	75.78
17	3341	241823.2	2025-06	72.38
18	3481	267581.7	2025-07	76.87
19	3341	251541.4	2025-08	75.29
20	3289	245382.5	2025-09	74.61
21	3436	249492.8	2025-10	72.61
22	3363	248512.4	2025-11	73.90
23	3539	264193.4	2025-12	74.65

```
In [7]: plt.figure(figsize=(15,5))  
sns.lineplot(  
    data=mau,  
    x='period',  
    y='active_users',  
    color='#6A5ACD',  
    linewidth=2.5  
)
```

```
plt.title('Месячная динамика активности пользователей (MAU)')
plt.xlabel('Дата')
plt.ylabel('Количество пользователей')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Вывод: Исходя из графика, можно сделать вывод, что пик аудитории достигается в августе и в декабре – это объясняется летними каникулами и зимними праздниками. После Нового года количество пользователей уменьшается, что связано с выходом на работу и учебу. Для того, чтобы избежать это спад, стоит выпускать новинки в этот период.

4. Конверсия поиска по устройствам

Выясним, с каких устройств пользователи чаще всего ищут фильмы и насколько успешно эти поиски заканчиваются реальным просмотром.

```
In [8]: device = pd.read_sql("""
        SELECT device_type,
               COUNT(search_id) AS total_searches,
               COUNT(clicked_result_position) AS clicked_searches,
               ROUND((COUNT(clicked_result_position)::NUMERIC / COUNT(search_id)) * 100, 2) AS conversion
        FROM search_logs
        GROUP BY device_type
        ORDER BY conversion DESC;""", engine)
device
```

```
Out[8]:
```

	device_type	total_searches	clicked_searches	conversion
0	Laptop	6585	3263	49.55
1	Mobile	6563	3230	49.22
2	Tablet	6783	3286	48.44
3	Smart TV	6569	3172	48.29

Вывод: Эффективность поиска практически одинакова на всех типах устройств. Конверсия из поискового запроса в клик стабильно держится

в районе 48–49%. Лидируют ноутбуки, а Smart TV незначительно отстает, это говорит о том, что поисковый интерфейс платформы одинаково удобен для пользователей на любых устройствах. Поскольку интерфейс работает стабильно, то стоит сфокусироваться на улучшении алгоритмов предложенных рекомендаций.

5. Скользящее среднее 7 дней

Найдем сглаженный недельный тренд просмотров, чтобы очистить данные от хаотичных скачков активности в выходные дни.

```
In [9]: rolling_7_days = pd.read_sql("""
        SELECT watch_date,
               SUM(watch_duration_minutes) AS day_value,
               ROUND(AVG(SUM(watch_duration_minutes)) OVER(
                   ORDER BY watch_date ROWS BETWEEN 6 PRECEDING AND CURRENT
               FROM watch_history
               GROUP BY watch_date
               ORDER BY watch_date ASC;""", engine)
        rolling_7_days
```

```
Out[9]:
```

	watch_date	day_value	rolling_7_day_avg
0	2024-01-01	7341.2	7341.2
1	2024-01-02	9906.9	8624.1
2	2024-01-03	7228.2	8158.8
3	2024-01-04	7352.4	7957.2
4	2024-01-05	7433.8	7852.5
...	...	...	...
726	2025-12-27	8739.9	8935.2
727	2025-12-28	6679.1	8805.3
728	2025-12-29	9811.5	9075.5
729	2025-12-30	8077.0	9018.6
730	2025-12-31	7600.9	8564.4

731 rows x 3 columns

```
In [10]: import matplotlib.ticker as ticker
        plt.figure(figsize=(12, 6))
        sns.lineplot(
            data=rolling_7_days,
            x='watch_date',
            y='day_value',
            label='Минуты за день',
```

```

        color='#B0BEC5',
        alpha=0.3
    )
    sns.lineplot(
        data=rolling_7_days,
        x='watch_date',
        y='rolling_7_day_avg',
        label='7-дневное скользящее среднее',
        color='#6A5ACD',
        linewidth=2.5
    )
    plt.title('Скользящее среднее просмотров за неделю')
    plt.xlabel('Дата')
    plt.ylabel('Время просмотра, минуты')
    plt.gca().xaxis.set_major_locator(ticker.MaxNLocator(12))
    plt.xticks(rotation=45)
    plt.legend()
    plt.tight_layout()
    plt.show()

```



Вывод: На графике видны пики роста — моменты, когда Netflix выпускает премьеры, а люди возвращаются к экранам.

6. ТОП-10 жанров

Найдем, какие жанры наиболее популярны

```

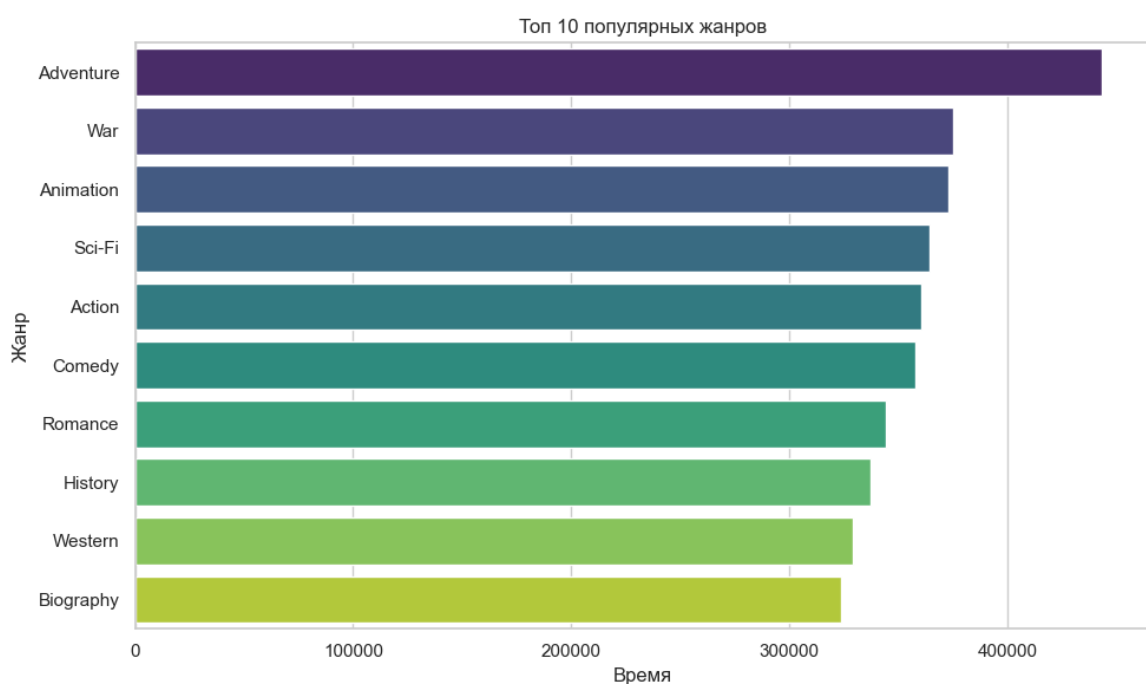
In [11]: genre=pd.read_sql("""
        SELECT m.genre_primary,
               COUNT(DISTINCT wh.user_id) AS unique_watch,
               SUM(wh.watch_duration_minutes) AS total_minutes
        FROM watch_history wh
        INNER JOIN movies m ON wh.movie_id = m.movie_id
        GROUP BY m.genre_primary
        ORDER BY total_minutes DESC
        LIMIT 10;""", engine)
genre

```

Out[11]:

	genre_primary	unique_watch	total_minutes
0	Adventure	4914	443280.8
1	War	4349	375281.4
2	Animation	4461	373216.8
3	Sci-Fi	4286	364505.4
4	Action	4353	360255.7
5	Comedy	4418	357961.7
6	Romance	3988	344227.4
7	History	4071	337278.4
8	Western	4208	329155.8
9	Biography	4016	323790.5

```
In [12]: plt.figure(figsize=(10,6))
sns.barplot(
    data=genre,
    x='total_minutes',
    y='genre_primary',
    palette='viridis',
    hue='genre_primary',
    legend=False
)
plt.title('Топ 10 популярных жанров')
plt.xlabel('Время')
plt.ylabel('Жанр')
plt.tight_layout()
plt.show()
```



Вывод: Лидером по времени просмотра стали Приключения, это показывает, что аудитория платформы предпочитает динамичный и зрелищный контент. Тогда, при планировании бюджетов на производство контента стоит делать фокус именно на приключенческие экшны.

7. Когортный анализ удержания пользователей

Посмотрим, как долго пользователь активен после месяца регистрации на платформе.

```
In [13]: cohort = pd.read_sql("""
WITH user_cohorts AS (
    SELECT DISTINCT user_id,
           subscription_start_date::date AS cohort_start
    FROM users
),
user_activities AS (
    SELECT DISTINCT user_id,
           watch_date::date AS activity_date
    FROM watch_history
)
SELECT
    TO_CHAR(uc.cohort_start, 'YYYY-MM') AS period,
    (EXTRACT(YEAR FROM AGE(ua.activity_date, uc.cohort_start)) * 12
     + EXTRACT(MONTH FROM AGE(ua.activity_date, uc.cohort_start))) AS
    lifetime_month,
    COUNT(DISTINCT uc.user_id) AS active_users
FROM user_cohorts uc
JOIN user_activities ua ON uc.user_id = ua.user_id
WHERE ua.activity_date >= uc.cohort_start
GROUP BY TO_CHAR(uc.cohort_start, 'YYYY-MM'), lifetime_month
ORDER BY period ASC, lifetime_month ASC;
""", engine='trino')
cohort
```

Out[13]:

	period	lifetime_month	active_users
0	2022-08	16.0	56
1	2022-08	17.0	91
2	2022-08	18.0	99
3	2022-08	19.0	96
4	2022-08	20.0	93
...	...	...	...
709	2025-07	5.0	44
710	2025-08	1.0	5
711	2025-08	2.0	4
712	2025-08	3.0	1
713	2025-08	4.0	1

714 rows x 3 columns

Вывод: Анализ данных показывает, что удержание пользователей стабильно. Количество активных зрителей внутри групп не падает до нуля со временем, что говорит хорошо о удержании аудитории.

8. Темп роста MoM

Посчитаем процентный прирост или падение активной аудитории (MAU) каждого месяца по отношению к предыдущему.

```
In [14]: mau = pd.read_sql("""
WITH monthly_mau AS (
    SELECT
        COUNT(DISTINCT user_id) AS active_users,
        TO_CHAR(watch_date::timestamp, 'YYYY-MM') AS year_month
    FROM watch_history
    GROUP BY TO_CHAR(watch_date::timestamp, 'YYYY-MM')
)
SELECT
    year_month,
    active_users,
    ROUND((active_users - LAG(active_users, 1) OVER (ORDER BY year_mo
        / LAG(active_users, 1) OVER (ORDER BY year_month) * 100, 2) AS
FROM monthly_mau;""", engine)
mau
```

Out[14]:

	year_month	active_users	growth_pct
0	2024-01	3372	NaN
1	2024-02	3384	0.36
2	2024-03	3380	-0.12
3	2024-04	3387	0.21
4	2024-05	3551	4.84
5	2024-06	3445	-2.99
6	2024-07	3398	-1.36
7	2024-08	3551	4.50
8	2024-09	3323	-6.42
9	2024-10	3434	3.34
10	2024-11	3331	-3.00
11	2024-12	3539	6.24
12	2025-01	3424	-3.25
13	2025-02	3118	-8.94
14	2025-03	3403	9.14
15	2025-04	3373	-0.88
16	2025-05	3438	1.93
17	2025-06	3341	-2.82
18	2025-07	3481	4.19
19	2025-08	3341	-4.02
20	2025-09	3289	-1.56
21	2025-10	3436	4.47
22	2025-11	3363	-2.12
23	2025-12	3539	5.23

9. ТОП-3 фильма внутри каждого жанра

Найдем тройку лидеров фильмов в внутри каждого жанра, чтобы сформировать список рекомендаций для новых пользователей

```
In [15]: rating = pd.read_sql("""  
        WITH movie_rating AS (
```

```

SELECT genre_primary,
       title,
       ROUND(AVG(r.rating)::NUMERIC,1) AS average_
       DENSE_RANK() OVER(PARTITION BY genre_primar
FROM movies m
JOIN reviews r ON r.movie_id = m.movie_id
GROUP BY genre_primary, title
)
SELECT genre_primary,
       title,
       average_rating,
       top_rating
FROM movie_rating
WHERE top_rating <= 3
ORDER BY genre_primary ASC, top_rating ASC;""", engine)
rating

```

Out[15]:

	genre_primary	title	average_rating	top_rating
0	Action	Legend Queen	4.1	1
1	Action	Little Warrior	4.1	2
2	Action	Dark Love	4.1	3
3	Adventure	Story Fire	4.2	1
4	Adventure	Family Kingdom	4.2	2
...	...	...	...	...
69	Western	House House	4.0	2
70	Western	Hero Mystery	4.0	2
71	Western	Journey Mystery	4.0	2
72	Western	Empire War	4.0	2
73	Western	King Warrior	4.0	3

74 rows × 4 columns

10. Финансовый профиль тарифных планов

Посчитаем количество подписчиков и средний чек для каждого тарифного плана и оценим вклад каждого сегмента в общую выручку.

```

In [16]: finance = pd.read_sql("""
SELECT COUNT(DISTINCT user_id),
       subscription_plan,
       ROUND(AVG(monthly_spend)::NUMERIC, 1) AS average_sp
FROM users
GROUP BY subscription_plan

```

```
ORDER BY average_spend DESC;''''', engine)
finance
```

Out[16]:

	count	subscription_plan	average_spend
0	3522	Standard	24.3
1	3507	Premium	21.7
2	1966	Basic	20.6
3	1005	Premium+	20.5

Вывод: Основную массу подписчиков генерируют тарифы Standard и Premium. Поскольку данные были созданы искусственно, парадоксально, что тариф Standard является самым высоким на платформе, опережая Premium.

11. Влияние качества трансляции на оценки

Посмотрим, как качество трансляции влияет на средние оценки.

```
In [17]: film_quality = pd.read_sql("""
SELECT quality,
        COUNT(review_id) AS total_review,
        ROUND(AVG(r.rating)::NUMERIC, 2) AS average_rating
FROM reviews r
JOIN watch_history wh ON wh.user_id = r.user_id AND wh.movie_id = r
GROUP BY quality
ORDER BY average_rating DESC;''''', engine)
film_quality
```

Out[17]:

	quality	total_review	average_rating
0	Ultra HD	8	4.63
1	SD	19	3.74
2	HD	66	3.53
3	4K	40	3.40

Вывод: Самую высокую среднюю оценку получили просмотры в Ultra HD, при этом формат 4K показал худший результат на платформе. Это говорит о том, что при переходе на тяжелый 4K-контент пользователи сталкиваются с техническими проблемами, из-за чего занижают оценки фильмам.

12. Анализ оттока пользователей

Найдем пользователей, которые не посмотрели контента за декабрь 2025 года, чтобы локализовать зоны риска потери выручки.

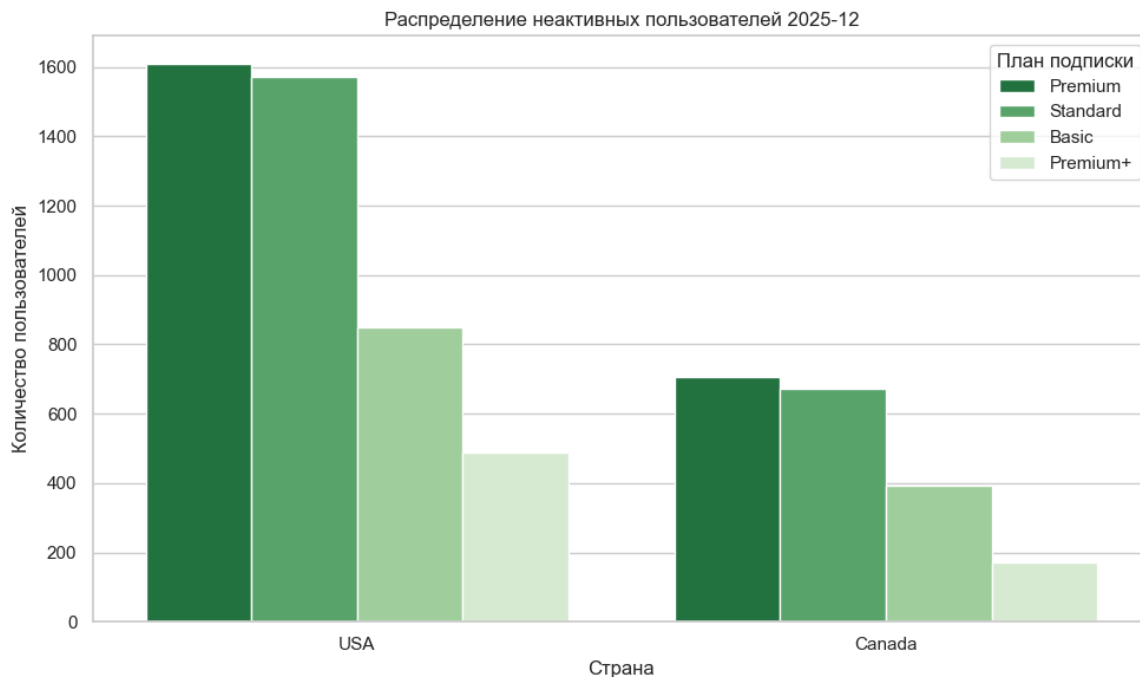
```
In [18]: subplan = pd.read_sql("""
SELECT COUNT(DISTINCT u.user_id) AS user_count,
        u.country,
        u.subscription_plan
FROM users u
LEFT JOIN watch_history wh ON wh.user_id = u.user_id

WHERE wh.user_id IS NULL
GROUP BY u.subscription_plan, u.country
ORDER BY user_count DESC;""", engine)
subplan
```

```
Out[18]:
```

	user_count	country	subscription_plan
0	1609	USA	Premium
1	1572	USA	Standard
2	849	USA	Basic
3	707	Canada	Premium
4	672	Canada	Standard
5	487	USA	Premium+
6	393	Canada	Basic
7	172	Canada	Premium+

```
In [19]: plt.figure(figsize=(10,6))
sns.barplot(
    data=subplan,
    x='country',
    y='user_count',
    hue='subscription_plan',
    palette='Greens_r'
)
plt.title('Распределение неактивных пользователей 2025-12')
plt.xlabel('Страна')
plt.ylabel('Количество пользователей')
plt.legend(title='План подписки')
plt.tight_layout()
plt.show()
```



Вывод: Основная масса "неактивных" платящих пользователей сосредоточена на самых дорогих тарифах — Premium и Standard, причём в США этот показатель превышает 1500 человек на каждый тариф. Чтобы уменьшить потенциальный отток, стоит предложить им новинки фильмов и сериалов, отправить персональные подборки и напомнить о преимуществах их подписки.

Заключение

В ходе работы было развёрнуто аналитическое хранилище данных (DWH) на базе PostgreSQL и спроектирована ER-диаграмма на основе датасета Netflix 2025:User Behavior Dataset (210K+ Records). Главные выводы и рекомендации, которые хотелось бы подчеркнуть:

- Смена географии:** Основной приток новых пользователей теперь идет из регионов — лидируют Северная Каролина и Теннесси, следовательно стоит перераспределить рекламный бюджет для средних городов.
- Сезонный спад:** Каждый год в феврале активность пользователей резко падает, стоит перенести даты выхода новых сезонов сериалов на этот месяц, чтобы вернуть людей к экранам.
- Технические неполадки:** Интерфейс поиска работает хорошо на любом устройстве, однако обнаружена проблема с форматом 4K. Стоит ускорить загрузку и уменьшить количество зависаний.
- Отток пользователей:** Выявлена масса "неактивных" платящих пользователей на дорогих тарифах, которые не посмотрели ни одной минуты за декабрь. Стоит запустить рассылку с подборками новинок, чтобы не потерять аудиторию.